Homage Current App Build

Architecture and feature memo - generated 2026-07-04

This memo summarizes the current local-first Homage build as represented in the working repo. It is written for product and technical review, not as a release note or legal document.

1. Build Overview

Homage is currently an Expo React Native app with a FastAPI backend. The app centers on outdoor, place-based play: map quests, melody and photo collection, sky confirmation, creative studios, local artifacts, multiplayer rooms, and local account/device pairing.

Layer	Current Build
Mobile/web app	Expo SDK 54, React 19, React Native 0.81, expo-router routes under app/app.
Backend	FastAPI service in backend/server.py with quest, upload, multiplayer, document, song, local database, and pairing endpoints.
Local data	SQLite database at backend/data/homage.sqlite3 via backend/local_database.py, plus filesystem folders under backend/data.
Mapping	react-native-maps in native app, a web shim for export, OSM/Overpass data for environment quests, and procedural fallback logic.
Sky	Camera + location + compass + motion overlay using local catalog data, Astronomy Engine horizon calculations, persistent calibration, and sky-quality readouts.

2. App Screens and Workflows

- **Home:** navigation into Game, Creator, Song Studio, Sky, Admin, Artifacts, and other feature routes.

- **Game:** outdoor quest loop using current GPS, map targets, melody/photo/sky stages, artifact persistence, and multiplayer state when a room is active.

- **Sky:** constellation scanner with camera background, full-sky catalog layer, selected constellation focus, alignment controls, and outdoor signal readiness checks.

- **Creator / Document Studio / Song Studio:** local authoring surfaces that publish or keep user-created quests, PDFs, songs, and artifacts tied to GPS-confirmed places.

- **Admin:** local database status, content counts, latest drops, and local account/device pairing code generation.

- **Pair Device:** phone-side pairing code claim flow that links a device to the local account and stores pairing data in SQLite.

- **Artifacts:** local collection playback/viewing for saved photos, documents, and rendered chip songs.

3. Local-First Account and Pairing

The backend now owns a local SQLite database with schema metadata, local accounts, devices, players, quests, artifacts, sync events, and pairing codes. The admin dashboard creates a pairing code and the phone Pair Device screen claims it. In the last verification pass, the local pairing flow inserted one device row into SQLite.

Relevant endpoints:

```
GET /local/database
POST /local/accounts
GET /local/accounts
POST /local/devices
POST /local/pairing-codes
POST /local/pairing-codes/{code}/claim
```

4. Sky System

The sky system has moved from only custom sidereal math toward a more instrument-like model. Homage still uses its local bright-star and constellation data, but star altitude/azimuth is now calculated through Astronomy Engine with refraction. The app also captures observer elevation when available.

- **Inputs:** camera permission, precise location, compass heading, device motion, time, observer latitude/longitude/elevation, selected constellation, and local star catalog.

- **Projection:** horizontal star positions are projected into camera space using heading, altitude, roll, and calibrated horizontal/vertical FOV.

- **Calibration:** heading offset, altitude offset, and lens FOV are saved in AsyncStorage so alignment survives app restarts.

- **Readiness:** requires fresh GPS, location accuracy at or below 35 meters, at least three stable fixes, compass readiness, motion sensor availability, and explicit outside confirmation.

- **Sky quality:** the scanner now reports twilight state plus Moon illumination and altitude to explain why stars may be hard to see.

Practical meaning: the math is more reliable, but the phone compass remains the limiting factor. The new Align control is important because real-world compass readings can drift by many degrees.

5. Map and Quest Data

The app still uses react-native-maps for native map display, with a web shim for export. Quest generation uses the backend first, then client-side Overpass, then procedural fallback. Both backend and client apply safety filtering to avoid private, blocked, road-adjacent, or unsuitable destinations.

- **Backend quest generation:** backend/quests.py queries Overpass, normalizes features, filters safety, selects diverse destinations, and caches responses in memory.

- **Client fallback:** app/lib/environmentQuests.ts can query Overpass directly if the backend fails.

- **New local cache:** recent non-procedural OSM-derived quest responses are cached on-device for 24 hours. If backend and Overpass both fail, the app can reuse recent real map data before falling back to procedural quests.

- **Current map engine:** react-native-maps remains in place. A MapLibre migration is intentionally not included yet because it changes the native map stack and should be handled as a separate dev-client/build migration.

6. Song Studio

Song Studio is now a richer chip-song tracker. It still renders local WAV audio and stores songs as artifacts or publishes them through the backend, but the harmonic controls are broader.

- All seven diatonic chord roots are available for each bar.

- Per-bar chord quality supports triad, maj7, min7, dom7, half diminished, and full diminished.

- Per-bar extensions support 9, flat 9, sharp 9, 11, sharp 11, 13, and flat 13.

- The stored song model keeps chordDegrees and adds optional chordVoicings for backward compatibility with older saved songs.

7. Backend Capabilities

Area	Current Capability
Health/admin	Health checks, root HTML status page, filesystem data directory, local database status.
Quests	Environment quest generation, custom quest publishing/listing, OSM and procedural fallback logic.
Multiplayer	Room creation/joining, signal-hunt progress, hide-and-seek state, sky locks, shared completion.
Artifacts	Photo upload metadata, documents, songs, artifact unlock, local record and sync event logging.
Local account	SQLite local accounts, devices, pairing codes, account-device linking.

8. Current Verification Status

- **Backend tests:** previously passed: python3 -m unittest backend.test_quests backend.test_multiplayer backend.test_local_database.

- **Expo web export:** passed after the latest sky/map-data updates: npx expo export --platform web.

- **TypeScript:** npx tsc --noEmit still reports pre-existing unrelated errors in app/maze.tsx and state/quests.ts. The recent sky, song, pairing, and cache changes did not add new TypeScript failures.

- **Manual browser UI:** the in-app browser could not attach to local Expo pages during the previous pairing test, so pairing was verified through the backend endpoints and SQLite instead.

9. Known Technical Caveats

- The repo worktree has many existing uncommitted/untracked app and backend files; treat this as an active development branch.

- MapLibre/vector-tile migration remains a future native-build task, not a safe one-line dependency change.

- Sky overlay accuracy is now better mathematically, but real-world accuracy still depends on compass calibration, phone sensor quality, camera FOV calibration, and open-sky conditions.

- The local-first database exists and supports pairing, but broader sync conflict resolution and multi-device replication semantics are not yet a full product layer.

- Several package versions are behind Expo recommended SDK versions, and npm audit reports existing vulnerabilities that need separate triage.

10. Recommended Next Steps

Priority	Next Step	Reason
1	Fix pre-existing TypeScript issues in app/maze.tsx and state/quests.ts.	Restores TypeScript as a clean regression gate.

2	Manual device test of Sky scanner outside at night.	Validates compass/FOV calibration on real sensors.
3	Add a guided star alignment flow using a known bright star.	More usable than manual arrow nudges for users.
4	Design MapLibre migration plan for dev-client/native builds.	Unlocks vector tiles, custom layers, and better offline maps.
5	Define sync semantics for local-first account/device data.	Turns pairing from local registration into reliable multi-device behavior.

Primary code areas referenced: app/app/sky.tsx, app/sky/astronomy.ts, app/sky/projection.ts, app/sky/useSkyPose.ts, app/lib/environmentQuests.ts, app/lib/chipSong.ts, app/app/song-studio.tsx, backend/server.py, backend/quests.py, backend/local_database.py.